

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Tuition / Coaching Institute Management System

Project Type: Web-based Tuition Management System

Current Version: Prototype

Frontend: React.js

Build Tool: Vite

Data Storage in Prototype: Browser Local Storage

Currency: INR (₹)

1. Introduction

1.1 Purpose

The purpose of the **Tuition / Coaching Institute Management System** is to provide a centralized web-based platform for managing the daily operations of a coaching or tuition institute.

The system is designed to replace or reduce dependency on manual records and scattered tools by providing modules for:

- Student management
- Faculty management
- Course management
- Batch management
- Attendance management
- Timetable management
- Examination management
- Assignment management
- Results management
- Fee and payment management
- Reports and analytics
- Notifications
- Institute settings

The current implementation is a **functional frontend prototype** that demonstrates the expected user interface, workflows, role-based navigation, data operations, and local persistence. It is designed so that a backend and database can be integrated in a future production version.

2. Scope of the System

The system will serve as a centralized management platform for a coaching institute.

The prototype supports multiple types of users:

1. Administrator
2. Accountant
3. Teacher
4. Student
5. Receptionist

Each role has access to a different set of modules based on its responsibilities.

The system provides a dashboard-based interface where users can navigate between authorized modules.

Main Scope

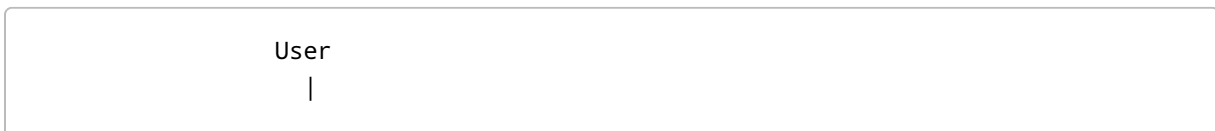
The system covers:

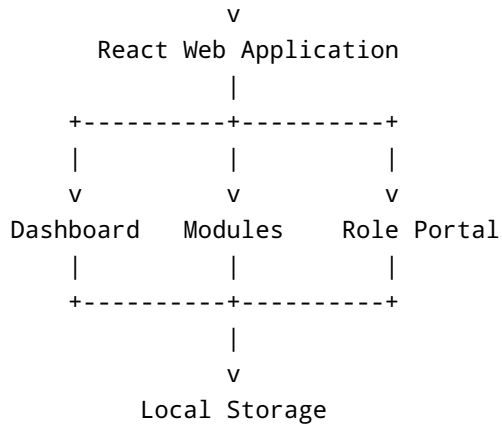
- Managing student records
- Managing faculty records
- Managing courses
- Managing batches
- Tracking attendance
- Maintaining class schedules
- Creating and managing examinations
- Creating assignments
- Recording examination results
- Recording and tracking fees
- Generating/viewing reports
- Displaying notifications
- Managing institute settings
- Providing role-specific portals

3. Product Perspective

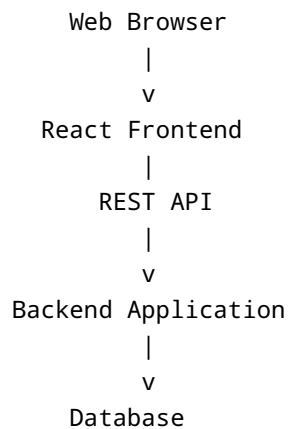
The system is designed as a single-page web application.

Current Prototype Architecture





Future Production Architecture



The current prototype intentionally operates without a backend and stores application data in browser `localStorage`.

4. User Roles

4.1 Administrator

The administrator has the highest level of access.

Accessible Modules

- Dashboard
- Students
- Faculty
- Courses

- Batches
- Attendance
- Timetable
- Exams
- Assignments
- Fees & Payments
- Reports
- Notifications
- Settings

Responsibilities

The administrator can manage institute-wide information and monitor overall academic and financial activity.

4.2 Accountant

The accountant primarily manages financial activities.

Accessible Modules

- Dashboard
- Batches
- Students
- Fees & Payments
- Reports
- Notifications

Responsibilities

- View student fee information
 - Record payments
 - Track pending fees
 - View financial reports
 - Review payment history
-

4.3 Teacher

The teacher has access to academic and batch-related information.

Accessible Modules

- Dashboard
- My Batches
- Students

- Attendance
- Timetable
- Exams
- Assignments
- Results
- Notifications

Responsibilities

- View assigned batches
 - View students in assigned batches
 - Mark attendance
 - View timetable
 - Create examinations
 - Enter examination results
 - Create assignments
 - Monitor academic information
-

4.4 Student

Students have access only to information relevant to themselves.

Accessible Modules

- Dashboard
- My Batch
- Attendance
- Fees & Payments
- Exams
- Assignments
- Results
- My Profile
- Notifications

Responsibilities

Students can:

- View their batch
- View attendance
- View fee status
- View payment history
- View examinations
- View assignments
- View examination results
- View personal profile information

- View notifications
-

4.5 Receptionist

The receptionist handles front-office operations.

Accessible Modules

- Dashboard
- Students
- Faculty
- Courses
- Batches
- Fees & Payments
- Notifications

Responsibilities

- Manage student information
 - View faculty information
 - View courses
 - Manage batch information
 - Handle fee-related operations
 - View notifications
-

5. Functional Requirements

5.1 Dashboard

The dashboard shall provide an overview of important institute information.

Requirements

The system shall display:

- Total number of students
- Faculty information
- Active batches
- Fee/revenue information
- Pending fees
- Attendance statistics
- Batch information
- Upcoming classes

- Notifications
- Quick actions
- Academic information

The dashboard shall allow users to navigate to relevant modules.

The information displayed shall depend on the logged-in role.

6. Student Management

The Student Management module shall maintain student records.

Student Information

The system shall maintain:

- Student ID
- Name
- Phone number
- Email
- Gender
- Date of birth
- Address
- Parent/guardian name
- Relationship
- Parent phone
- Emergency contact
- Course
- Batch
- Admission date
- Status
- Total fee
- Paid fee
- Attendance percentage

Functional Requirements

The system shall allow authorized users to:

- View students
- Search students
- Filter students
- Add students
- Edit student information
- Delete student records

- View student profiles
- View student fee information
- View attendance information
- View batch information

Student records shall contain a unique student ID.

7. Faculty Management

The Faculty module shall maintain information about teaching staff.

Faculty Information

The system shall store:

- Faculty ID
- Name
- Subject
- Qualification
- Experience
- Phone
- Email
- Salary
- Joining date
- Status
- Number of assigned batches

Functional Requirements

Authorized users shall be able to:

- View faculty
- Search faculty
- Add faculty
- Edit faculty
- Delete faculty
- View faculty details
- View assigned batches

The faculty profile shall show the batches assigned to the faculty member.

8. Course Management

The system shall maintain coaching courses.

Example courses include:

- JEE
- NEET
- Foundation
- Board Exams

Each course shall contain:

- Course ID
- Course name
- Description
- Duration
- Fee
- Status

Authorized users shall be able to:

- View courses
 - Add courses
 - Edit courses
 - Delete courses
 - View course details
-

9. Batch Management

The Batch Management module shall organize students into classes/batches.

Each batch shall contain:

- Batch ID
- Batch name
- Course
- Faculty
- Classroom
- Start date
- End date
- Start time
- End time
- Days

- Maximum capacity
- Status

Functional Requirements

The system shall allow authorized users to:

- Create batches
- Edit batches
- Delete batches
- View batches
- Search/filter batches
- View students belonging to a batch
- View assigned faculty
- View batch schedule
- Monitor batch capacity

The prototype also includes validation intended to prevent invalid batch scheduling information.

10. Attendance Management

The Attendance module shall allow attendance to be maintained for students.

Attendance statuses include:

- Present
- Late
- Absent

Authorized users, particularly teachers and administrators, shall be able to mark attendance.

The system shall display attendance information for students and batches.

Student attendance percentages shall be visible in relevant student and dashboard views.

The student portal shall only expose the student's own attendance information.

11. Timetable Management

The Timetable module shall display the weekly schedule.

The timetable shall contain:

- Batch
- Subject
- Faculty
- Days
- Time
- Classroom

The timetable shall allow users to understand when and where each batch is scheduled.

The prototype includes validation intended to prevent conflicts involving batch time and classroom information.

12. Examination Management

The Examination module shall allow authorized users to create and manage examinations.

Each examination shall contain:

- Examination ID
- Examination title
- Examination date
- Course
- Batch
- Subject
- Total marks
- Passing marks

Functional Requirements

Authorized users shall be able to:

- Create examinations
- Edit examinations
- View examinations
- Enter results
- View examination results

Validation shall ensure:

- Examination title is provided
- Examination date is provided
- Total marks are greater than zero
- Passing marks are greater than zero

- Passing marks do not exceed total marks
 - Student marks do not exceed total marks
-

13. Assignment Management

The Assignment module shall manage academic assignments.

Each assignment shall contain:

- Assignment ID
- Title
- Description
- Subject
- Batch
- Due date
- Marks
- Status

Assignment statuses include:

- Open
- Closed

Teachers and administrators shall be able to create and manage assignments.

Students shall be able to view assignments relevant to their batch.

14. Results Management

The Results module shall maintain examination marks.

Each result shall contain:

- Result ID
- Examination ID
- Student ID
- Marks

The system shall allow authorized users to enter and update student marks.

The system shall validate that marks:

- Are numeric

- Are not negative
- Do not exceed total examination marks

The system can determine grades based on percentage.

Grade Logic

90% and above	→ A+
80% - 89%	→ A
70% - 79%	→ B+
60% - 69%	→ B
50% - 59%	→ C
Below 50%	→ D

Students shall be able to view their own results.

15. Fees and Payments

The Fees & Payments module shall manage student fee information.

Each student shall have:

- Total fee
- Amount paid
- Pending amount

The system shall calculate:

Pending Fee = Total Fee - Paid Fee

Payment Information

Each payment shall contain:

- Receipt ID
- Student ID
- Date
- Amount
- Payment mode

Supported payment modes in the prototype include:

- Cash

- UPI
- Bank Transfer

Functional Requirements

Authorized users shall be able to:

- View student fee status
- Search students
- Record payments
- View payment history
- View pending fees
- Print payment receipts

The system shall prevent a payment from exceeding the student's pending fee.

16. Reports and Analytics

The Reports module shall provide summarized institute information.

The system shall provide information such as:

- Total revenue
- Pending fees
- Student count
- Average attendance
- Batch-wise student distribution
- Course-wise revenue
- Financial reports

Accountants shall receive financial-focused reports.

Administrators shall have broader institute-level reports.

The prototype provides options for viewing/printing/exporting report information.

17. Notifications

The Notification module shall display system notifications.

Notifications can include:

- Fee reminders

- Assignment announcements
- Examination announcements
- Other system alerts

Each notification contains:

- Notification ID
- Title
- Message
- Type
- Date

Notification types include:

- Warning
- Information
- Success

Users can mark all notifications as read.

18. Settings

The Settings module shall provide institute-level preferences.

The prototype supports:

- Institute name
- Currency display
- Saving institute settings
- Exporting application data
- Resetting demo data

The system can export the current local data as a JSON backup.

The prototype can also restore the initial demo dataset.

19. Student Portal

The student portal provides student-specific information.

A student can access:

My Batch

Displays:

- Course
- Batch
- Faculty
- Classroom
- Schedule
- Time
- Attendance

My Profile

Displays:

- Personal information
- Guardian information
- Academic information
- Attendance
- Fee information

My Fees

Displays:

- Total fee
- Paid amount
- Pending amount
- Payment history

Exams

Displays examinations relevant to the student's batch.

Assignments

Displays assignments relevant to the student's batch.

Results

Displays the student's own examination results.

20. Role-Based Access Control

The system implements role-based navigation and page access.

Each role has an allowed set of modules.

For example:

Admin

- └─ Students
- └─ Faculty
- └─ Courses
- └─ Batches
- └─ Attendance
- └─ Exams
- └─ Assignments
- └─ Fees
- └─ Reports
- └─ Settings

Teacher

- └─ My Batches
- └─ Students
- └─ Attendance
- └─ Exams
- └─ Assignments
- └─ Results

Student

- └─ My Batch
- └─ Attendance
- └─ Fees
- └─ Exams
- └─ Assignments
- └─ Results
- └─ My Profile

If a user attempts to access a page outside their allowed role, the application displays an **Access Denied** view.

Production Requirement

In the current prototype, role handling is implemented on the frontend.

For production, authorization must also be enforced on the backend using authenticated sessions/JWT and server-side authorization.

21. Data Management

The prototype uses browser `localStorage` for persistence.

Application data is stored under the local storage key:

```
tuition-erp-data
```

The selected role is also persisted locally.

This allows the prototype to retain changes after page refresh without requiring a backend.

The prototype can:

- Save changes
 - Reload saved data
 - Export JSON backup
 - Restore initial demo data
-

22. Navigation Requirements

The application shall provide:

- Sidebar navigation
- Role-specific navigation
- Active page indication
- Browser history support
- Back navigation
- Forward navigation
- Responsive sidebar behavior

The application uses URL hash-based navigation such as:

```
#/admin/dashboard  
#/admin/students  
#/teacher/attendance  
#/student/results
```

This provides a simple client-side routing mechanism for the prototype.

23. User Interface Requirements

The UI shall provide:

- Responsive layout
- Dashboard cards
- Data tables
- Forms
- Search fields
- Filters
- Buttons
- Status badges
- Progress indicators
- Modals
- Notifications/toasts
- Profile views
- Charts and visual summaries

The interface shall maintain consistent styling across all modules.

The system shall support desktop and smaller-screen layouts.

24. Validation Requirements

The system shall validate user input before saving information.

Examples include:

Student

Required student information shall be validated before saving.

Payment

```
Amount > 0
Amount <= Pending Fee
```

Examination

```
Total Marks > 0
Passing Marks > 0
Passing Marks <= Total Marks
```

Result

```
Marks >= 0
Marks <= Total Marks
```

Assignment

```
Title required
Due Date required
Batch required
Marks > 0
```

Invalid information shall generate an error notification.

25. Non-Functional Requirements

25.1 Performance

The application should load pages quickly and provide responsive interactions.

Since the prototype uses local data, operations should execute without significant network delay.

25.2 Usability

The application should be easy to understand for:

- Administrators
- Teachers
- Accountants
- Receptionists
- Students

Navigation and actions should use clear labels and consistent UI elements.

25.3 Security

The production system should provide:

- Secure authentication
- Password hashing

- JWT/session management
- Server-side authorization
- Input validation
- HTTPS
- Protection against unauthorized access
- Secure database access

The current prototype does not provide production-grade authentication because it is designed as a frontend prototype.

25.4 Reliability

The production system should preserve important institute data and provide appropriate backup and recovery mechanisms.

The current prototype provides a JSON data export and demo-data reset mechanism.

25.5 Scalability

The production architecture should support increasing numbers of:

- Students
- Faculty members
- Courses
- Batches
- Exams
- Assignments
- Payments

A backend database should replace local browser storage for multi-user operation.

26. Data Entities

The system currently models the following major entities:

Student

```
Student
- id
- name
- phone
- email
```

- gender
- dob
- address
- parentName
- relationship
- parentPhone
- emergency
- course
- batch
- admissionDate
- status
- fee
- paid
- attendance

Faculty

- Faculty
- id
 - name
 - subject
 - qualification
 - experience
 - phone
 - email
 - salary
 - joiningDate
 - status
 - batches

Course

- Course
- id
 - name
 - description
 - duration
 - fee
 - status

Batch

- Batch
- id

- name
- course
- faculty
- classroom
- startDate
- endDate
- startTime
- endTime
- days
- max
- status

Payment

- Payment
- id
 - studentId
 - date
 - amount
 - mode

Examination

- Exam
- id
 - title
 - date
 - course
 - batch
 - subject
 - total
 - passing

Result

- Result
- id
 - examId
 - studentId
 - marks

Assignment

```
Assignment
- id
- title
- description
- subject
- batch
- dueDate
- marks
- status
```

Notification

```
Notification
- id
- title
- text
- type
- date
```

27. Relationship Model

The conceptual relationships are:

```
Course
  |
  | 1:N
  v
Batch
  |
  | N:1
  v
Faculty

Batch
  |
  | 1:N
  v
Student
  |
  +-----> Payment
```



```

|
+-----> Result
|
+-----> Attendance

Batch
|
+-----> Exam
|
+-----> Assignment

Exam
|
| 1:N
v
Result

```

In the future database implementation, these relationships should be represented using proper primary keys and foreign keys.

28. Prototype Limitations

The current version is intentionally a prototype.

The following production features are not yet implemented as a real backend system:

- Server-side authentication
- Real user registration
- Password management
- Database persistence
- REST API layer
- Server-side authorization
- Multi-user concurrent access
- Cloud database
- Real email/SMS notifications
- Online payment gateway
- Production-grade audit logging
- Advanced reporting/export infrastructure

The current implementation uses demo data and browser `localStorage`.

29. Future Enhancements

The system can be extended with:

Backend

Develop a backend using technologies such as:

- Spring Boot
- REST APIs
- PostgreSQL/MySQL

Authentication

Implement:

- Login
- Registration
- JWT authentication
- Role-based authorization
- Password reset

Parent Portal

Add a dedicated parent portal for:

- Attendance
- Fees
- Results
- Assignments
- Notifications

Online Payments

Integrate payment gateways to allow students/parents to pay fees online.

Communication

Add:

- Email notifications
- SMS
- WhatsApp integration
- Push notifications

Advanced Analytics

Add:

- Student performance trends
- Attendance trends
- Fee collection trends
- Batch performance
- Course performance

Automated Reports

Generate:

- PDF fee receipts
- Student report cards
- Attendance reports
- Financial reports
- Performance reports

30. Technology Requirements

Current Prototype

Technology	Purpose
React.js	Frontend UI
React DOM	Rendering React application
Vite	Development/build tool
JavaScript	Application logic
CSS	Styling
Browser Local Storage	Local persistence

The project currently uses React 19 and Vite.

Future Production Stack

A possible production architecture is:

```
Frontend
React.js
  |
  | REST API
  v
Backend
Spring Boot
  |
  v
PostgreSQL
```

31. Deployment

The prototype is deployed as a web application and can be accessed through the Vercel deployment.

The deployment demonstrates how the frontend can be hosted and accessed through a browser without requiring local development setup.

32. System Workflow

Administrator Workflow

```
Admin Login
  ↓
Dashboard
  ↓
Manage Students
  ↓
Manage Courses
  ↓
Create Batches
  ↓
Assign Faculty
  ↓
Manage Attendance
  ↓
Manage Exams & Assignments
  ↓
Manage Fees
```

↓
View Reports

Teacher Workflow

Teacher
↓
Dashboard
↓
My Batches
↓
View Students
↓
Mark Attendance
↓
Create Assignment
↓
Create Exam
↓
Enter Results

Student Workflow

Student
↓
Dashboard
↓
My Batch
↓
View Attendance
↓
View Assignments
↓
View Exams
↓
View Results
↓
View Fees
↓
View Profile

33. Acceptance Criteria

The prototype shall be considered successful when:

1. Users can navigate through available modules.
 2. Different roles display different navigation options.
 3. Unauthorized pages display an access-denied state.
 4. Student records can be displayed and managed.
 5. Faculty records can be displayed and managed.
 6. Courses and batches can be managed.
 7. Attendance can be recorded.
 8. Timetable information can be displayed.
 9. Exams can be created and managed.
 10. Assignments can be created and managed.
 11. Results can be recorded and viewed.
 12. Fees can be recorded and tracked.
 13. Payment history can be viewed.
 14. Payment receipts can be printed.
 15. Reports can display summarized information.
 16. Notifications can be displayed and cleared.
 17. Settings can be updated.
 18. Data changes persist in browser local storage.
 19. Data can be exported as JSON.
 20. Demo data can be restored.
-

34. Conclusion

The Tuition / Coaching Institute Management System is a centralized web-based platform designed to digitize and simplify the management of coaching institute operations.

The current prototype demonstrates the major workflows required by different institute users, including administrators, accountants, teachers, students, and receptionists.

The prototype establishes the application's user experience, module structure, role-based access model, data model, validation rules, and operational workflows.

The next stage of development would be to replace the browser-based data layer with a secure backend and relational database, implement real authentication and authorization, and make the system suitable for simultaneous use by multiple users.

The prototype therefore acts as the **functional foundation and UI specification for the future production Tuition Management System.**